Fault Model

Our fault model for the calculator module is an extensive framework aimed at covering a range of potential issues to ensure the module operates both robustly and accurately. It focuses on three main areas: ensuring functional accuracy, adherence to protocols, and effective error management, with specific attention to:

Functional Accuracy: At the heart of our fault model is the assurance of the calculator module's functional accuracy. This includes verifying that basic arithmetic operations such as addition, subtraction, and bitwise AND are performed accurately. Our goal is to identify and address any errors that could result in incorrect outputs. This is labeled as “CORRECTNESS TEST” in my testbench

Adherence to Protocols: Another critical aspect of our model is ensuring that the calculator module follows the specified protocols, which outline the order and timing of its operations. We conduct thorough checks to identify any breaches or non-compliance that might interfere with its correct functioning.

Effective Error Management: Lastly, our model prioritizes effective error management, focusing on how the module handles exceptional circumstances, like stack overflows or the reception of unexpected signals like "DONE." We assess the module's ability to detect and correct errors, aiming for resilient error handling and recovery processes.Thus, the testbench purposely triggers errors and tests if error outputs are signaled at the current clock signal.

What could go wrong?

1) **Stack Overflow:**

In this fault model, we evaluate the module's response to situations where the stack might overflow due to too many elements being added. This evaluation is performed in the testbench through the use of a "stackOverflow_error" assertion. By pushing more elements onto the stack than it can handle, we simulate a potential overflow scenario. The critical aspect of this test is verifying whether the "stackOverflow" flag is appropriately activated, which would demonstrate the module's capability to recognize and manage overflow conditions.

2) **Unexpected Done:**

The "DONE" signal is intended to indicate the completion of a calculation. Nevertheless, due to possible design or implementation errors, it might be triggered prematurely, resulting in inaccurate results. To address this, the testbench employs the "prop_unexpectedDone" property

to examine the occurrence of these premature "DONE" signals. By creating conditions under which the "DONE" signal might erroneously activate, the testbench assesses whether the "unexpectedDone" flag is correctly set, ensuring the system's ability to detect and signal unexpected completions.

3) Data Overflow

The module operates with a specific bit-width for data processing. The fault model addresses situations where input data surpasses this bit-width, focusing on how the system handles data overflows. To test this, the testbench introduces temporary values that deliberately exceed the defined bit-width, through addition or subtraction, to trigger the "dataOverflow" error. Then, we check whether the dataOverflow flag is triggered.This approach allows for the evaluation of the module's capacity to manage and respond to overflow conditions effectively.

4) Invalid Protocol

Our calculator operates according to a precise sequence of operations, outlined by a specific protocol. Deviations from this established protocol could lead to significant functional issues. The fault model addresses these concerns by focusing on adherence to the operation sequence. It utilizes various properties such as "prop_protocol_2_element_operations," prop_protocol_2," and "op_decrease_stack" to scrutinize the calculator's compliance with the protocol. These properties are designed to identify deviations, including unauthorized operations or incorrect order of operations. Assertions linked with these properties are responsible for activating the "protocolError" flag whenever such discrepancies are detected, confirming the module's capability to enforce and adhere to the correct operational protocol.

5) Computation Error:

The calculator is designed to accurately execute fundamental arithmetic operations such as addition, subtraction, AND, among others. However, flaws in these operations could cause erroneous outcomes. To mitigate this risk, the testbench includes a range of test scenarios for these arithmetic functions, utilizing randomly generated data inputs. It employs assertions to confirm the accuracy of the outcomes, comparing the actual results of the calculator with the anticipated results to ensure their correctness.

Faults each phase:

Phase	Errors
0	- No errors
1 (1 fault expected)	- **Correctness Error in computation:** The result of **adding** two numbers is 1 greater than the expected result. For example, when I add 5 and 3 on the stack, the result in this calculator is 9 instead of 8.
2 (2 faults expected)	- **Swapping Error:** The result of swapping two numbers on the stack is incorrect. They remain at the same position. For example, I push two elements onto an empty stack and swap the elements in the next clock cycle. However, the elements remain at the same position. - **Missing Data Overflow:** Although I did some test operations that resulted in an overflow with the pre-defined 16 bit-width, the dataOverflow flag was not raised as expected. I triggered overflows using subtraction and addition; however, both methods did not successfully raise the dataOverflow signal as expected.
3 (1 fault expected)	- **Correct Signal Missed/Not Asserted:** Despite the program finishing with no errors, the `correct` signal is not asserted as expected. This suggests that there could be correctness issues within this calculator's operations.
4 (2 faults expected)	- **StackOverflow Error:** : The "stackOverflow" error condition is not correctly triggered when the stack overflows. - **Protocol Error:** When the stack is empty, it unexpectedly does not cause a protocol error.
5 (0 fault expected)	- No errors
6 (3 faults expected)	- **Protocol Violation:** Initiating a start action followed by another start does not lead to a protocol error as it should. - **Swapping Error:** The "swap" operation mistakenly reduces the stack size by 1, causing improper stack management and failing to trigger an error correctly. - **Stack Management:** Once the stack expands to hold 5 elements, it starts to drop values.
7 (1 fault expected)	- **Adding Error:** The calculator does not properly manage carryover when the least significant 12 bits are summed, resulting in overflow problems.